

# **Exp 10: OLED Display Interfacing with Raspberry Pi Pico**

**Name: Gobinda Dey  
CDSID: GDEY**



## Table of Contents

| S.No. |                         | Page No |
|-------|-------------------------|---------|
| 1     | Introduction            | 3       |
| 2     | Scope                   | 3       |
| 3     | Prerequisite            | 3       |
| 4     | Connection Diagram      | 3       |
| 5     | Working Principle       | 4       |
| 6     | Application Code        | 4       |
| 7     | Output & Hardware Image | 5       |
| 8     | Appendix                | 5       |

## 1. Introduction:

- A **16×2 LCD display** is a commonly used output device in embedded systems for displaying alphanumeric characters. It is based on the **HD44780 controller**, which allows communication in either 8-bit or 4-bit mode.
- In this experiment, the LCD is interfaced with the Raspberry Pi Pico using **4-bit mode**, reducing the number of GPIO pins required. The system displays a predefined message (name) on the LCD.

## 2. Scope:

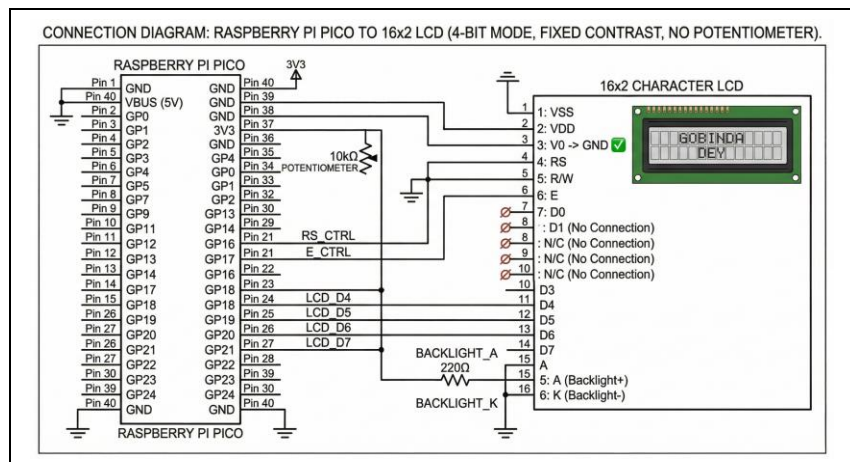
- To understand interfacing of LCD with microcontroller
- To learn GPIO control and timing requirements
- To implement 4-bit communication protocol
- To display user-defined messages on LCD

## 3. Prerequisite:

**Hardware:** Raspberry Pi Pico Microcontroller, USB Connector, Breadboard, 16×2 LCD Display

**Software:** Visual studio code, Pico SDK, CMake & GCC Toolchain

## 4. Connection Diagram:



## 5. Working Principle:

The LCD operates using the HD44780 controller, which accepts commands and data through GPIO pins.

- RS (Register Select):
  - 0 → Command mode
  - 1 → Data mode
- E (Enable):
  - A HIGH-to-LOW pulse triggers data read
- 4-bit Mode Operation:
  - Data is sent in two nibbles (4 bits each)
  - Reduces required pins from 8 → 4
- Initialization Sequence:
  - LCD is configured for:
    - 4-bit mode
    - 2-line display
    - Cursor settings
- Contrast Control:
  - V0 is connected to GND → fixed contrast

## 6. Application code:

```
#include "pico/stdlib.h"

// Pin definitions
#define RS 0
#define EN 1
#define D4 2
#define D5 3
#define D6 4
#define D7 5

void lcd_send_nibble(uint8_t nibble) {
    gpio_put(D4, (nibble >> 0) & 1);
    gpio_put(D5, (nibble >> 1) & 1);
    gpio_put(D6, (nibble >> 2) & 1);
    gpio_put(D7, (nibble >> 3) & 1);

    gpio_put(EN, 1);
    sleep_us(1);
    gpio_put(EN, 0);
    sleep_us(100);
}

void lcd_send_byte(uint8_t data, int rs) {
    gpio_put(RS, rs);
    lcd_send_nibble(data >> 4);
    lcd_send_nibble(data & 0x0F);
}

void lcd_cmd(uint8_t cmd) {
    lcd_send_byte(cmd, 0);
    sleep_ms(2);
}
```

```
void lcd_char(char c) {
    lcd_send_byte(c, 1);
}

void lcd_string(const char *str) {
    while (*str) {
        lcd_char(*str++);
    }
}

void lcd_init() {
    gpio_init(RS); gpio_set_dir(RS, GPIO_OUT);
    gpio_init(EN); gpio_set_dir(EN, GPIO_OUT);
    gpio_init(D4); gpio_set_dir(D4, GPIO_OUT);
    gpio_init(D5); gpio_set_dir(D5, GPIO_OUT);
    gpio_init(D6); gpio_set_dir(D6, GPIO_OUT);
    gpio_init(D7); gpio_set_dir(D7, GPIO_OUT);

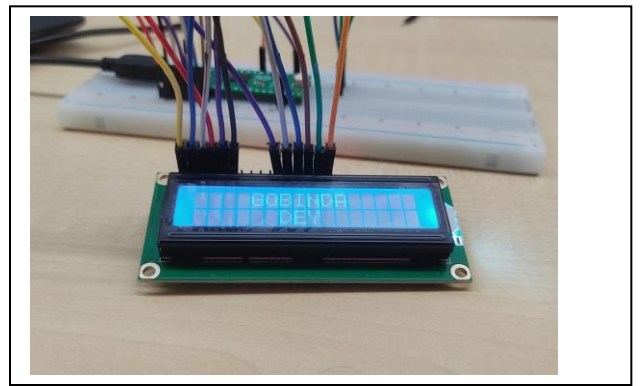
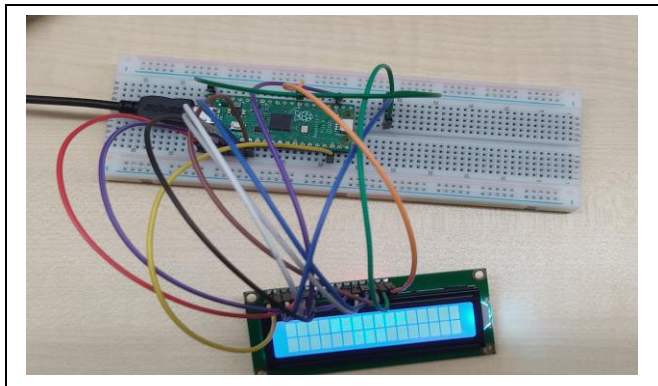
    sleep_ms(50);

    lcd_send_nibble(0x03);
    sleep_ms(5);
    lcd_send_nibble(0x03);
    sleep_ms(5);
    lcd_send_nibble(0x03);
    sleep_ms(5);
    lcd_send_nibble(0x02);

    lcd_cmd(0x28); // 4-bit, 2 line
    lcd_cmd(0x0C); // display ON
    lcd_cmd(0x06); // cursor increment
    lcd_cmd(0x01); // clear
}
```

```
int main() {  
    stdio_init_all();  
    lcd_init();  
  
    // Line 1 (row 0, column 4)  
    lcd_cmd(0x80 + 4);  
    lcd_string("GOBINDA");  
  
    // Line 2 (row 1, column 6)  
    lcd_cmd(0xC0 + 6);  
    lcd_string("DEY");  
  
    while (1);  
}
```

## 7. Output & Hardware Image:



## 8. Appendix:

### References

YouTube and Other Online Sources

[Official Documentation for Raspberry Pi Pico and Pico W](#)

[GitHub Repository for Raspberry Pi Pico SDK](#)